Files në Python

Leksion

Alion Emini

kurse.informatika
Kurs i Programimit në Python

October 1, 2025

Përmbajtja

1. Hyrje: pse na duhen skedarët?
2. Llojet e skedarëve dhe aksesimi
3. Emrat e skedarëve dhe objektet file
4. Hapja e skedarit: `open()`
5. Shkrimi i të dhënave në skedar
6. Leximi i të dhënave nga skedari
7. Bashkimi i `\n` me një string
8. Heqja e `\n`
9. Shtimi në fund me `'a'`
10. Numrat në skedarë
11. Cikle me skedarë dhe EOF
12. Records (registre)
13. Përdorimi i `with` (context manager)

Objektivat e kapitullit

- Shpjegosh pse duhen skedarët dhe hapat bazë *hap—proceso—mbyll*.
- Dallosh llojet e skedarëve (tekst vs. binar) dhe metodat e aksesit (sekuencial vs. direkt).
- Përdorësh `open(filename, mode)` me modalitetet `'r'`, `'w'`, `'a'` dhe të specifikosh rrugë (path) të skedarëve.
- Shkruash dhe lexosh të dhëna tekstuale; të menaxhosh `\n` dhe `rstrip()`.
- Shkruash/lexosh numra (konvertimi `str()`, `int()`, `float()`).
- Përdorësh cikle `while` / `for` për të procesuar skedarë dhe për të zbuluar EOF.
- Strukturash të dhënat si *record-e* me *field-e* dhe t'i lexosh/shkruash në mënyrë të qëndrueshme.
- Përdorësh `with` (context manager) për hapje/mbyllje automatike dhe më të sigurt të skedarëve.

Përmbajtja

1. Hyrje: pse na duhen skedarët?
2. Llojet e skedarëve dhe aksesimi
3. Emrat e skedarëve dhe objektet file
4. Hapja e skedarit: `open()`
5. Shkrimi i të dhënave në skedar
6. Leximi i të dhënave nga skedari
7. Bashkimi i `\n` me një string
8. Heqja e `\n`
9. Shtimi në fund me `'a'`
10. Numrat në skedarë
11. Cikle me skedarë dhe EOF
12. Records (registre)
13. Përdorimi i `with` (context manager)

Pse skedarë?

- Variablat në RAM zhduken pas ekzekutimit; skedarët ruajnë të dhënat *midis* ekzekutimeve.
- Programe tipike që përdorin skedarë: përpunues teksti, redaktues imazhesh, spreadsheet, lojëra (save/load), shfletues web (cookies).

Përkufizime bazë

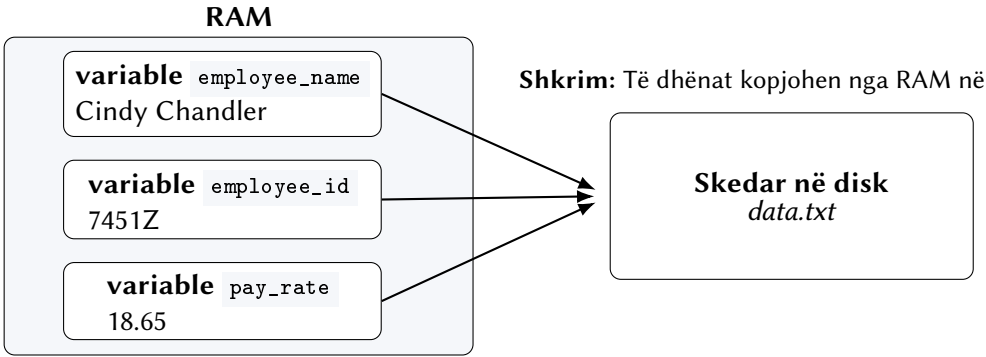
Output file

Shkrimi i të dhënave nga RAM-i në skedar (dalje).

Input file

Leximi i të dhënave nga skedari në RAM (hyrje).

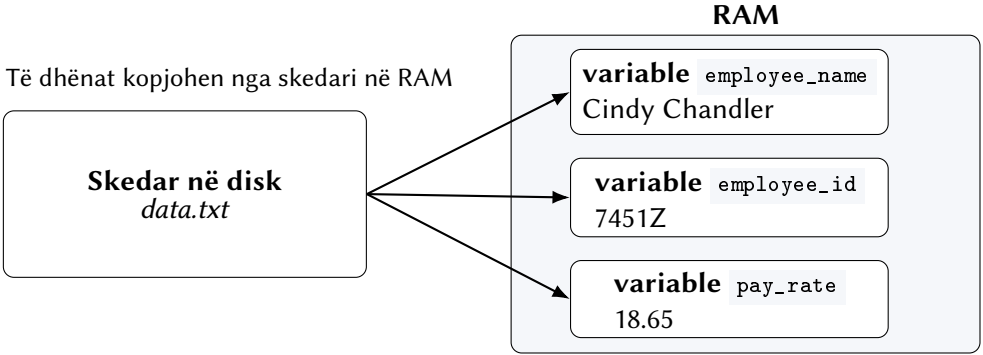
Skemë: Shkrimi i të dhënave (output)



Shkrim: Të dhënat kopjohen nga RAM në skedar

Skemë: Leximi i të dhënave (input)

Lexim: Të dhënat kopjohen nga skedari në RAM



Përmbajtja

1. Hyrje: pse na duhen skedarët?
2. Llojet e skedarëve dhe aksesimi
3. Emrat e skedarëve dhe objektet file
4. Hapja e skedarit: `open()`
5. Shkrimi i të dhënave në skedar
6. Leximi i të dhënave nga skedari
7. Bashkimi i `\n` me një string
8. Heqja e `\n`
9. Shtimi në fund me `'a'`
10. Numrat në skedarë
11. Cikle me skedarë dhe EOF
12. Records (registre)
13. Përdorimi i `with` (context manager)

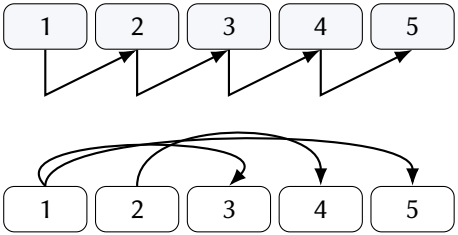
Llojet: tekst dhe binar

- **Skedarë teksti**: ASCII/Unicode; të lexueshëm me editor; numrat si karaktere.
- **Skedarë binarë**: për programe/formate (`.png`, `.pdf`); jo të lexueshëm si tekst.

Akses sekuencial vs. direkt

Akses sekuencial: të dhënat lexohen/shkruhen njëra pas tjetrës, në radhë.
Akses direkt: mund të kërcesh direkt te çdo pjesë e skedarit (zakonisht në skedarë binarë).

sekuencial: kalon sipas radhës

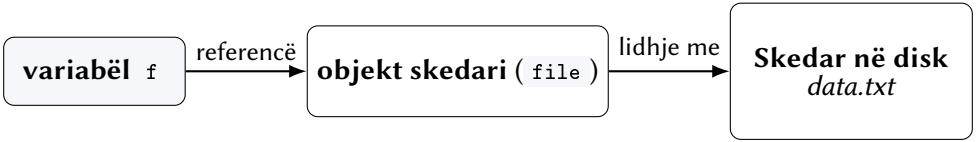


Direkt: kërcim në cilëndo pjesë

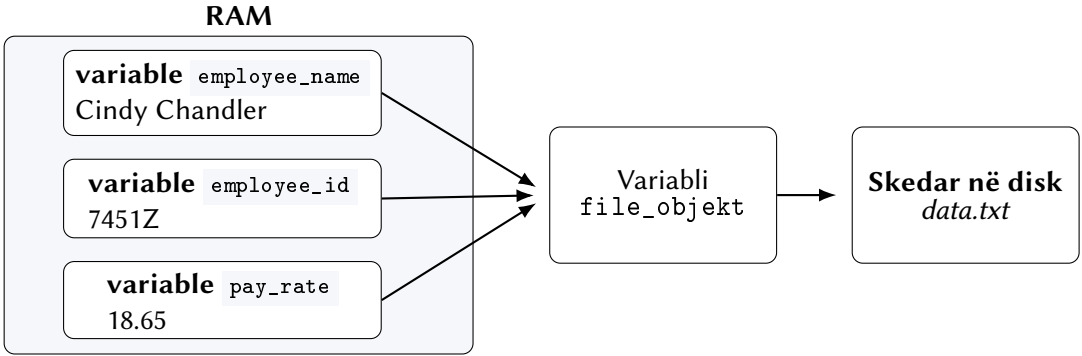
Përmbajtja

1. Hyrje: pse na duhen skedarët?
2. Llojet e skedarëve dhe aksesimi
3. Emrat e skedarëve dhe objektet file
4. Hapja e skedarit: `open()`
5. Shkrimi i të dhënave në skedar
6. Leximi i të dhënave nga skedari
7. Bashkimi i `\n` me një string
8. Heqja e `\n`
9. Shtimi në fund me `'a'`
10. Numrat në skedarë
11. Cikle me skedarë dhe EOF
12. Records (registre)
13. Përdorimi i `with` (context manager)

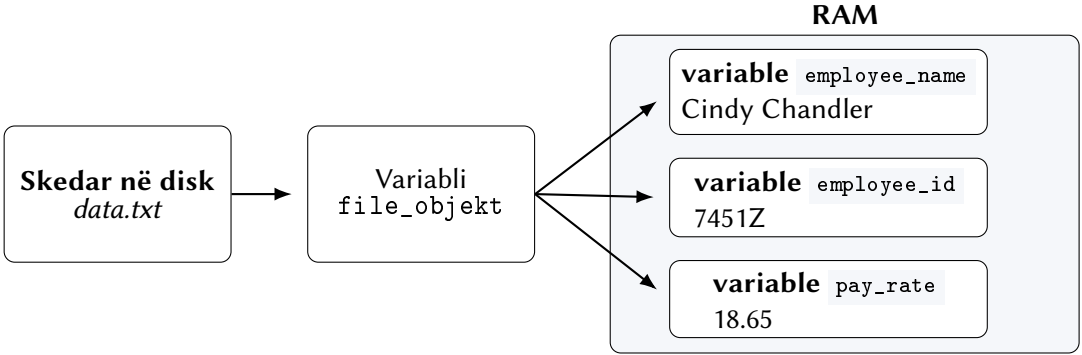
Variabël → file object → skedar



Skemë: Shkrimi i të dhënave (output)



Skemë: Leximi i të dhënave (input)



Përmbajtja

1. Hyrje: pse na duhen skedarët?
2. Llojet e skedarëve dhe aksesimi
3. Emrat e skedarëve dhe objektet file
4. Hapja e skedarit: `open()`
5. Shkrimi i të dhënave në skedar
6. Leximi i të dhënave nga skedari
7. Bashkimi i `\n` me një string
8. Heqja e `\n`
9. Shtimi në fund me `'a'`
10. Numrat në skedarë
11. Cikle me skedarë dhe EOF
12. Records (registre)
13. Përdorimi i `with` (context manager)

Sintaksa dhe termat

```
file_variable = open(filename, mode)
```

- `file_variable` — variabla që referon *file object*-in.
- `filename` — emri i skedarit si string.
- `mode` — mënyra e hapjes (`'r'` , `'w'` , `'a'`).

Modalitetet kryesore

Modaliteti	Përshkrimi
`'r'`	Vetëm **lexim**.
`'w'`	**Shkrim**. Nëse ekziston, **fshihet përmbajtja**; nëse mungon, **krijohet**.
`'a'`	**Shtim në fund**. Nëse mungon, **krijohet**.

Shembull: hapja për lexim/shkrim

```python
# Hap "customers.txt" për lexim ('r')
customer_file = open('customers.txt', 'r')

# Hap ose krijon "sales.txt" për shkrim ('w')
sales_file = open('sales.txt', 'w')
```

Kujdes

Hapja me `'w'` e zbraz skedarin ekzistues.

Specifikimi i rrugës (path)

Nëse jep vetëm emër skedari, përdoret folderi i programit. Për rrugë absolute në Windows përdor `r'...'` (raw string).

```
test_file = open('test.txt', 'w')

test_file = open(r'C:\Users\Blake\temp\test.txt', 'w')
```

Këshillë

Në Windows përdor `r'...'` që `\` të mos trajtohen si escape. Në Linux/macOS përdor `'/home/user/...'` ose `'/Users/user/...'`.

Përmbajtja

1. Hyrje: pse na duhen skedarët?
2. Llojet e skedarëve dhe aksesimi
3. Emrat e skedarëve dhe objektet file
4. Hapja e skedarit: `open()`
5. Shkrimi i të dhënave në skedar
6. Leximi i të dhënave nga skedari
7. Bashkimi i `\n` me një string
8. Heqja e `\n`
9. Shtimi në fund me `'a'`
10. Numrat në skedarë
11. Cikle me skedarë dhe EOF
12. Records (registre)
13. Përdorimi i `with` (context manager)

Funksion vs. Metodë

Shpjegim

- **Funksioni** është një bllok kodi që mund të thirret në mënyrë të pavarur dhe zakonisht merr argumente si hyrje (`len(obj)`).
- **Metoda** është një funksion që i përket një objekti të caktuar dhe thirret përmes këtij objekti (`obj.upper()`).

Shembull

- Pra, `len('abc')` është funksion i përgjithshëm, ndërsa `'abc'.upper()` është metodë e objektit `str` .

write()

Përkufizim

`file.write(string)` — shkruan *string*-un në skedar (`'w'` ose `'a'`).

```
customer_file = open('customers.txt', 'w')

customer_file = open('customers.txt', 'a')
```

Çfarë kthen metoda `write()`?

Metoda `write()` kthen një **numër të plotë** që tregon sa karaktere u shkruan në skedar.

```
n = customer_file.write('Charles Pace')
print(n) # -> 12 (numri i karaktereve të shkruara)
```

Programi 1 — file_write.py

```python
def main():
    # Hape skedarin "philosophers.txt" për shkrim.
    outfile = open('philosophers.txt', 'w')

    # Shkruaj tre emra filozofësh (secili me \n në fund).
    row1 = outfile.write('John Locke\n')
    row2 = outfile.write('David Hume\n')
    row3 = outfile.write('Edmund Burke\n')

    # Mbyll skedarin.
    outfile.close()

    # Printimi i numrit të karaktereve të shkruara në skedar.
    print(f'Numri i karaktereve të shkruara në rreshtin e parë: {row1}')
    print(f'Numri i karaktereve të shkruara në rreshtin e dytë: {row2}')
    print(f'Numri i karaktereve të shkruara në rreshtin e tretë: {row3}')
```

Programi 1 — Output i `file_write.py`

Përmbajtja e skedarit `philosophers.txt`:

```
John Locke
David Hume
Edmund Burke
```

Output:

```
Numri i karaktereve të shkruara në rreshtin e parë: 12
Numri i karaktereve të shkruara në rreshtin e dytë: 11
Numri i karaktereve të shkruara në rreshtin e tretë: 13
```

Shënim

Çdo emër është në një rresht më vete, sepse përdorëm `\n` në fund të secilit string.

\n në skedar

Çfarë ruhet?

```
John Locke\nDavid Hume\nEdmund Burke\n
```

Pse duhet të mbyllim skedarin?

Rëndësia e mbylljes së skedarit

Kur programi përfundon punën me një skedar, duhet ta mbyllë atë me `close()`. Kjo:

- Shkëput lidhjen me skedarin.
- Siguron që të gjitha të dhënat e pambetura në buffer të shkruhen në disk.
- Parandalon humbjen e të dhënave nëse programi mbyllet papritur.

Në disa sisteme, nëse nuk mbyllet skedari, të dhënat mund të mos ruhen fizikisht në disk.

```
1 customer_file.close()
```

Përmbajtja

1. Hyrje: pse na duhen skedarët?
2. Llojet e skedarëve dhe aksesimi
3. Emrat e skedarëve dhe objektet file
4. Hapja e skedarit: `open()`
5. Shkrimi i të dhënave në skedar
6. Leximi i të dhënave nga skedari
7. Bashkimi i `\n` me një string
8. Heqja e `\n`
9. Shtimi në fund me `'a'`
10. Numrat në skedarë
11. Cikle me skedarë dhe EOF
12. Records (registre)
13. Përdorimi i `with` (context manager)

`read()` dhe `readline()`

- `read()` — lexon të gjithë përmbajtjen si një *string*.
- `readline()` — lexon një rresht (përfshin `\n`, nëse ekziston).

Programi 2 — file_read.py - Përdorimi i read()

```
def main():
    # Hap skedarin për lexim.
    infile = open('philosophers.txt', 'r')

    # Lexo të gjithë përmbajtjen si një string.
    file_contents = infile.read()

    # Mbyll skedarin.
    infile.close()

    # Shtyp përmbajtjen e lexuar.
    print(file_contents)
```

Programi 2 — Output i `file_read.py`

Output:

```
John Locke
David Hume
Edmund Burke
```

Shënim

`print(file_contents)` shfaq të gjithë tekstin si një string të vetëm, duke ruajtur rreshtat origjinalë.

`readline()` — rresht pas rreshti

Vërejtje

`readline()` kthen rreshtin *me* `\n`. `print()` shton një `\n` tjetër.

Programi 3 — line_read.py - Përdorimi i readline()

```
def main():
    infile = open('philosophers.txt', 'r')

    # Lexo tre rreshta nga skedari.
    line1 = infile.readline()
    line2 = infile.readline()
    line3 = infile.readline()

    # Mbyll skedarin.
    infile.close()

    # Kujdes: çdo line përmban edhe '\n'.
    print(line1) # shton një '\n' nga print, mund të duket si rresht bosh
    print(line2)
    print(line3)
```

Programi 3 — Output i `line_read.py`

Output:

```
John Locke

David Hume

Edmund Burke
```

Shënim

Çdo `print(line)` shton një rresht bosh shtesë, sepse `line` përmban `\n` dhe `print()` shton një tjetër.

Pozicioni i leximit (1)

Pozicioni i leximit

```
John Locke\nDavid Hume\nEdmund Burke\n
```

Pozicioni i leximit (2)

Pozicioni i leximit

```
John Locke\nDavid Hume\nEdmund Burke\n
```

Pozicioni i leximit (3)

Pozicioni i leximit

```
John Locke\nDavid Hume\nEdmund Burke\n
```

Përmbajtja

1. Hyrje: pse na duhen skedarët?
2. Llojet e skedarëve dhe aksesimi
3. Emrat e skedarëve dhe objektet file
4. Hapja e skedarit: `open()`
5. Shkrimi i të dhënave në skedar
6. Leximi i të dhënave nga skedari
7. Bashkimi i `\n` me një string
8. Heqja e `\n`
9. Shtimi në fund me `'a'`
10. Numrat në skedarë
11. Cikle me skedarë dhe EOF
12. Records (registre)
13. Përdorimi i `with` (context manager)

Programi 4 — write_names.py

```python
def main():
    print('Shkruani emrat e tre shokëve.')
    name1 = input('Shoku #1: ')
    name2 = input('Shoku #2: ')
    name3 = input('Shoku #3: ')

    # Hape skedarin friends.txt për shkrim.
    myfile = open('friends.txt', 'w')

    # Shkruaji emrat në skedar (secilin në një rresht).
    myfile.write(name1 + '\n')
    myfile.write(name2 + '\n')
    myfile.write(name3 + '\n')

    myfile.close()
    print('Emrat u shkruan në friends.txt.')
```

Program 4 — Output i write_names.py

Output:

```
Shkruani emrat e tre shokëve.
Shoku \#1: Joe
Shoku \#2: Rose
Shoku \#3: Geri
Emrat u shkruan në friends.txt.
```

Përmbajtja e friends.txt

```
Joe
Rose
Geri
```

Këshillë: përdorimi i *f-strings*

- `myfile.write(f'{name1}\n')`
- `myfile.write(f'{name2}\n')`
- `myfile.write(f'{name3}\n')`

Përmbajtja

1. Hyrje: pse na duhen skedarët?
2. Llojet e skedarëve dhe aksesimi
3. Emrat e skedarëve dhe objektet file
4. Hapja e skedarit: `open()`
5. Shkrimi i të dhënave në skedar
6. Leximi i të dhënave nga skedari
7. Bashkimi i `\n` me një string
8. Heqja e `\n`
9. Shtimi në fund me `'a'`
10. Numrat në skedarë
11. Cikle me skedarë dhe EOF
12. Records (registre)
13. Përdorimi i `with` (context manager)

rstrip() për të larguar \n

```
name = 'Joanne Manchester\n'
name = name.rstrip('\n') # -> 'Joanne Manchester'
```

Shënim

`rstrip(chars)` heq nga ana e djathtë karakteret e listuara; pa argument, heq hapësira të bardha (përfshirë `\n`, `\r`, `\t`).

Programi 5 — strip_newline.py

```python
def main():
    infile = open('philosophers.txt', 'r')

    line1 = infile.readline()
    line2 = infile.readline()
    line3 = infile.readline()

    # Hiq '\n' nga fundi i secilit string (nëse ekziston).
    line1 = line1.rstrip('\n')
    line2 = line2.rstrip('\n')
    line3 = line3.rstrip('\n')

    infile.close()

    # Shfaq të dhënat pa rreshta bosh shtesë.
    print(line1)
    print(line2)
    print(line3)
```

Programi 5 — Output i strip_newline.py

Output:

```
John Locke
David Hume
Edmund Burke
```

Shënim

Nuk ka rreshta bosh shtesë, sepse `rstrip('\n')` heq karakterin e ri-rreshtimit nga fundi i çdo string-u.

Përmbajtja

1. Hyrje: pse na duhen skedarët?
2. Llojet e skedarëve dhe aksesimi
3. Emrat e skedarëve dhe objektet file
4. Hapja e skedarit: `open()`
5. Shkrimi i të dhënave në skedar
6. Leximi i të dhënave nga skedari
7. Bashkimi i `\n` me një string
8. Heqja e `\n`
9. Shtimi në fund me `'a'`
10. Numrat në skedarë
11. Cikle me skedarë dhe EOF
12. Records (registre)
13. Përdorimi i `with` (context manager)

Append: shto pa fshirë përmbajtjen ekzistuese

```
myfile = open('friends.txt', 'a')
myfile.write('Matt\n')
myfile.write('Chris\n')
myfile.close()
```

`friends.txt` para:

```
Joe
Rose
Geri
```

`friends.txt` pas:

```
Joe
Rose
Geri
Matt
Chris
```

Kujdes

`'w'` *zbraz* skedarin; `'a'` *shton* në fund.

Programi 6 — file_append.py

```python
def main():
    # Hap një skedar të quajtur "philosophers.txt".
    outfile = open('philosophers.txt', 'a')

    # Shkruan emrat e tre filozofëve në skedar.
    outfile.write('John Locke\n')
    outfile.write('David Hume\n')
    outfile.write('Edmund Burke\n')

    # Mbyll skedarin.
    outfile.close()

    print('The names were appended to philosophers.txt.')
```

Programi 6 — Output i file_append.py

Përmbajtja e skedarit philosophers.txt para ekzekutimit

```
John Locke
David Hume
Edmund Burke
```

Përmbajtja e skedarit philosophers.txt pas ekzekutimit

```
John Locke
David Hume
Edmund Burke
John Locke
David Hume
Edmund Burke
```

Përmbajtja

1. Hyrje: pse na duhen skedarët?
2. Llojet e skedarëve dhe aksesimi
3. Emrat e skedarëve dhe objektet file
4. Hapja e skedarit: `open()`
5. Shkrimi i të dhënave në skedar
6. Leximi i të dhënave nga skedari
7. Bashkimi i `\n` me një string
8. Heqja e `\n`
9. Shtimi në fund me `'a'`
10. Numrat në skedarë
11. Cikle me skedarë dhe EOF
12. Records (registre)
13. Përdorimi i `with` (context manager)

Shkrimi i numrave në skedar tekst

Vërejtje

Metoda `write()` mund të shkruajë *vetëm string-ë* në një skedar teksti. Numrat duhet të konvertohen më parë në string me funksionin `str()`.

- Nëse `num = 99`, atëherë `str(num)` kthen `'99'`.
- Shprehja `str(num1) + '\n'` konverton vlerën e `num1` në string dhe i bashkon `\n` në fund (p.sh. prodhon `'22\n'`).
- E njëjta gjë bëhet për `num2` dhe `num3`.

Programi 7 — write_numbers.py

```python
def main():
    outfile = open('numbers.txt', 'w')

    num1 = int(input('Shkruaj një numër: '))
    num2 = int(input('Shkruaj një numër tjetër: '))
    num3 = int(input('Shkruaj edhe një numër: '))

    # Shkruaji numrat (secili në rresht më vete).
    outfile.write(str(num1) + '\n')
    outfile.write(str(num2) + '\n')
    outfile.write(str(num3) + '\n')

    outfile.close()
    print('Të dhënat u shkruan në numbers.txt')
```

Programi 7 — Output i `write_numbers.py`

Output

```
Shkruaj një numër: 22
Shkruaj një numër tjetër: 17
Shkruaj edhe një numër: 5
Të dhënat u shkruan në numbers.txt.
```

Përmbajtja e skedarit `numbers.txt`

```
22
17
5
```

Leximi i numrave dhe konvertimi

```
infile = open('numbers.txt', 'r')
value = infile.readline() # p.sh. '22\n'
infile.close()

infile = open('numbers.txt', 'r')
value = int(infile.readline()) # -> 22 (int injoron '\n')
infile.close()
```

Shënim

`int()` dhe `float()` injorojnë `\n`, por hedhin `ValueError` nëse string-u nuk është numerik.

Programi 8 — read_numbers.py

```python
def main():
    infile = open('numbers.txt', 'r')

    # Lexo tre rreshta dhe konverto në int.
    num1 = int(infile.readline())
    num2 = int(infile.readline())
    num3 = int(infile.readline())

    infile.close()

    # Llogarit totalin.
    total = num1 + num2 + num3

    # Shfaq.
    print(f'The numbers are: {num1}, {num2}, {num3}')
    print(f'Their total is: {total}')
```

Programi 8 — Output i `read_numbers.py`

Output:

```
The numbers are: 22, 17, 5
Their total is: 44
```

Shënim

Numrat lexohen nga skedari, konvertohen në `int` dhe i gjendet totali.

Përmbajtja

1. Hyrje: pse na duhen skedarët?
2. Llojet e skedarëve dhe aksesimi
3. Emrat e skedarëve dhe objektet file
4. Hapja e skedarit: `open()`
5. Shkrimi i të dhënave në skedar
6. Leximi i të dhënave nga skedari
7. Bashkimi i `\n` me një string
8. Heqja e `\n`
9. Shtimi në fund me `'a'`
10. Numrat në skedarë
11. Cikle me skedarë dhe EOF
12. Records (registre)
13. Përdorimi i `with` (context manager)

Pse cikle me skedarë?

Arsyet kryesore

- Automatizim: një bllok kodi për shumë rreshta.
- Shkallëzim: i njëjti kod funksionon për 3 apo 3000 rreshta.
- Llogaritje/Kërkim: përpunim sistematik i të dhënave.

Programi 9 — write_sales.py

```
def main():
    num_days = int(input('Për sa ditë ke shitje? '))
    sales_file = open('sales.txt', 'w')

    for count in range(1, num_days + 1):
        sales = float(input(f'Fut shitjet për ditën #{count}: '))
        sales_file.write(f'{sales}\n')

    sales_file.close()
    print('Të dhënat u shkruan në sales.txt.')
```

Programi 9 — Output i `write_sales.py`

Output:

```
Për sa ditë ke shitje? 3
Fut shitjet për ditën #1: 100.50
Fut shitjet për ditën #2: 200.75
Fut shitjet për ditën #3: 150.25
Të dhënat u shkruan në sales.txt.
```

Përmbajtja e `sales.txt`

```
100.5
200.75
150.25
```

Çfarë është EOF dhe *priming read*?

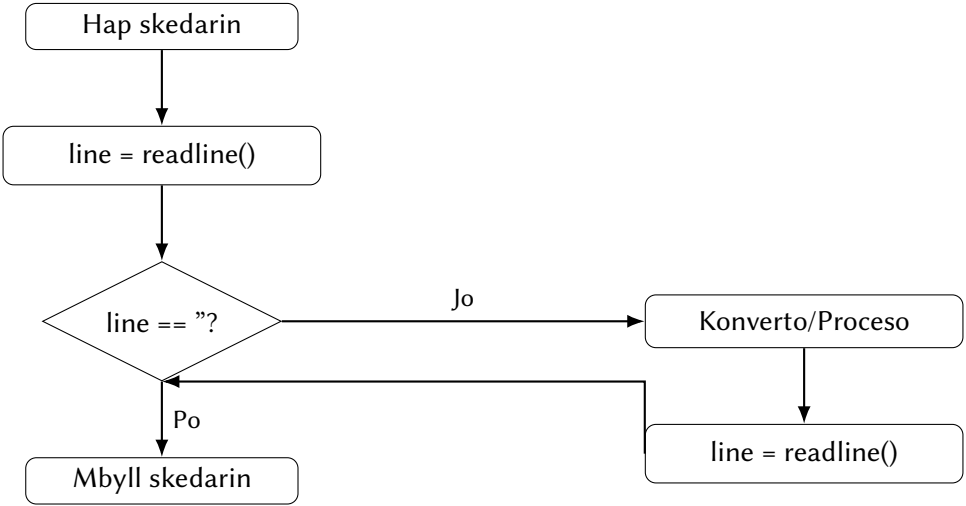
Përkufizim: EOF

End Of File — s'ka më të dhëna për t'u lexuar. `readline()` kthen `''`.

Përkufizim: Priming read / cikli sentinel

Lexo një vlerë *para* hyrjes në `while`; përdoret si kusht. Brenda ciklit përpuno dhe lexo rreshtin tjetër.

Diagramë rrjedhe: *priming read* me `while`



Programi 10 — read_sales.py

```python
def main():
    sales_file = open('sales.txt', 'r')

    line = sales_file.readline() # priming read
    while line != '':
        amount = float(line)
        print(f'{amount:.2f}')
        line = sales_file.readline()

    sales_file.close()
```

Kujdes

`readline()` kthen *string*; ktheje me `float(...)` ose `int(...)` përpara llogaritjeve.

Programi 10 — Output i `read_sales.py`

Output:

```
100.50
200.75
150.25
```

Shënim

Çdo rresht i skedarit `sales.txt` lexohet, konvertohet në `float` dhe shfaqet me dy shifra pas presjes.

Programi 11 — read_sales2.py (iterim direkt)

```python
def main():
    sales_file = open('sales.txt', 'r')
    for line in sales_file:
        print(f'{float(line):.2f}')
    sales_file.close()

if __name__ == '__main__':
    main()
```

Programi 11 — Output i `read_sales2.py`

Output:

```
100.50
200.75
150.25
```

Shënim

Përdorimi i `for line in file` lexon çdo rresht automatikisht deri në fund të skedarit, pa nevojë për kontroll manual të EOF.

`for` apo `while`? — Përmbledhje teorike

- `for line in file` — më i thjeshtë për lexim të plotë.
- `while` + *priming read* — fleksibël kur ndalon mbi kushte të veçanta.
- Me `for` zbulimi i EOF është automatik; me `while` është manual.

Përmbajtja

1. Hyrje: pse na duhen skedarët?
2. Llojet e skedarëve dhe aksesimi
3. Emrat e skedarëve dhe objektet file
4. Hapja e skedarit: `open()`
5. Shkrimi i të dhënave në skedar
6. Leximi i të dhënave nga skedari
7. Bashkimi i `\n` me një string
8. Heqja e `\n`
9. Shtimi në fund me `'a'`
10. Numrat në skedarë
11. Cikle me skedarë dhe EOF
12. Records (registre)
13. Përdorimi i `with` (context manager)

Record dhe field

Përkufizime

Record = set i plotë fushash për një njësi (p.sh. punonjës).
Field = copë e vetme info (emër, ID, departament).

Skemë: fusha brenda një record-i

`'Ingrid Virgo\n'`	*Name*
`'4587\n'`	*ID number*
`'Engineering\n'`	*Department*

Record
(Punonjës)

Në skedar, fushat ruhen *rresht pas rreshti*.

```
Ingrid Virgo
4587
Engineering
```

Disa record-e rresht pas rreshti

Record 1

```
'Ingrid Virgo\n'
'4587\n'
'Engineering\n'
```

Record 2

```
'Julia Rich\n'
'4588\n'
'Research\n'
```

Record 3

```
'Greg Young\n'
'4589\n'
'Marketing\n'
```

Skedar me record-e punonjësish

```
Ingrid Virgo
4587
Engineering
Julia Rich
4588
Research
Greg Young
4589
Marketing
```

Teori: Organizimi i të dhënave në skedarë

Mesazh kyç

Përdor strukturë të qëndrueshme (të njëjtat fusha, i njëjti rend). Nëse mungon një fushë, *sinkronizimi* prishet dhe leximet pasuese dalin gabim.

Programi 12 — save_emp_records.py

```python
def main():
    num_emps = int(input('Sa record-e punonjësish do të krijosh? '))

    emp_file = open('employees.txt', 'w')

    # Për secilin punonjës, merr të dhënat dhe shkruaji si 3 rreshta.
    for count in range(1, num_emps + 1):
        print(f'Fut të dhënat për punonjësin #{count}')
        name = input('Name: ')
        id_num = input('ID number: ')
        dept = input('Department: ')

        emp_file.write(f'{name}\n')
        emp_file.write(f'{id_num}\n')
        emp_file.write(f'{dept}\n')

        print() # rresht bosh

    emp_file.close()
    print('Record-et u ruajtën në employees.txt.')
```

Programi 12 — Output i save_emp_records.py I

Output:

```
Sa record-e punonjësish do të krijosh? 2
Fut të dhënat për punonjësin #1
Name: Ingrid Virgo
ID number: 4587
Department: Engineering

Fut të dhënat për punonjësin #2
Name: Julia Rich
ID number: 4588
Department: Research

Record-et u ruajtën në employees.txt.
```

Programi 12 — Output i `save_emp_records.py`

Përmbajtja e `employees.txt`

```
Ingrid Virgo
4587
Engineering
Julia Rich
4588
Research
```

Rëndësi e renditjes së fushave

Shënim

Renditja në **shkrim** duhet të përputhet saktësisht me renditjen në **lexim**: $name \rightarrow id \rightarrow dept$.

Programi 13 — read_emp_records.py I

```
def main():
    emp_file = open('employees.txt', 'r')
    name = emp_file.readline().rstrip('\n')

    while name != '':
        id_num = emp_file.readline().rstrip('\n')
        dept = emp_file.readline().rstrip('\n')

        print(f'Name: {name}')
        print(f'ID: {id_num}')
        print(f'Dept: {dept}')
        print()

        name = emp_file.readline().rstrip('\n')

```

Programi 13 — read_emp_records.py II

```
16    emp_file.close()
```

Programi 13 — Output i `read_emp_records.py`

Output:

```
Name: Ingrid Virgo
ID: 4587
Dept: Engineering

Name: Julia Rich
ID: 4588
Dept: Research
```

Shënim

Çdo record lexohet nga tre rreshta radhazi dhe shfaqet me formatim të qartë.

Gabime të zakonshme me record-e

Kujdes

- Fusha të munguar/a tepruara ⇒ prishet sinkronizimi.
- Rreshta bosh të padëshiruar ⇒ përdor `rstrip('\n')`.
- Konvertimet numerike vetëm kur fusha është numerike.

Përmbajtja

1. Hyrje: pse na duhen skedarët?
2. Llojet e skedarëve dhe aksesimi
3. Emrat e skedarëve dhe objektet file
4. Hapja e skedarit: `open()`
5. Shkrimi i të dhënave në skedar
6. Leximi i të dhënave nga skedari
7. Bashkimi i `\n` me një string
8. Heqja e `\n`
9. Shtimi në fund me `'a'`
10. Numrat në skedarë
11. Cikle me skedarë dhe EOF
12. Records (registre)
13. Përdorimi i `with` (context manager)

Çfarë është një context manager?

Ideja

Menaxhon burime gjatë një blloku: hap në hyrje, *mbyll automatikisht* në dalje — edhe nëse ndodh gabim.

Pse i rëndësishëm?

- Mbyllje e garantuar e skedarëve.
- Mbrojtje ndaj *exception*-eve.
- Kod më i pastër dhe i shkurtër.

Sintaksa bazë e `with`

```
1 with open('emri_skedarit.txt', 'w') as file:
2     file.write('Përshëndetje!')
3 # Pas daljes nga blloku, skedari mbyllet automatikisht
```

Programi 14 — `write_with.py`

```python
def main():
    with open('philosophers.txt', 'w', encoding='utf-8') as f:
        f.write('John Locke\n')
        f.write('David Hume\n')
        f.write('Edmund Burke\n')
```

Program 14 — Output i `write_with.py`

Output:

```
John Locke
David Hume
Edmund Burke
```

Shënim

Çdo emër është në një rresht më vete, sepse përdorëm `\n` në fund të secilit string.

Programi 15 — `with_read.py`

```
def main():
    with open('sales.txt', 'r', encoding='utf-8') as f:
        for line in f:
            amount = float(line)
            print(f'{amount:.2f}')
```

Programi 15 — Output i `with_read.py`

Output:

```
100.50
200.75
150.25
```

Shënim

Çdo rresht i skedarit `sales.txt` lexohet, konvertohet në `float` dhe shfaqet me dy shifra pas presjes.

Programi 16 — with_multiple.py

```
def replace_department(emp_id, new_dept):
    with open('employees.txt', 'r', encoding='utf-8') as src, \
         open('employees.tmp', 'w', encoding='utf-8') as dst:

        while True:
            name = src.readline()
            if name == '':
                break
            id_num = src.readline()
            dept = src.readline()

            name = name.rstrip('\n')
            id_num = id_num.rstrip('\n')
            dept = dept.rstrip('\n')

            if id_num == emp_id:
                dept = new_dept

            dst.write(f'{name}\n{id_num}\n{dept}\n')
```

Programi 16 — `with_multiple.py`

```
1 def main():
2     replace_department('4587', 'HR')
3     replace_department('4588', 'Finance')
4     replace_department('4789', 'IT')
```

Programi 16 — Output i `with_multiple.py` I

Shembull:

Nëse thërrasim `replace_department('4588', 'Finance')`, atëherë në `employees.tmp` rekordi me ID 4588 do të ketë departamentin e ndryshuar në IT.

`employees.txt` **para:**

```
Ingrid Virgo
4587
Engineering
Julia Rich
4588
Research
```

`employees.tmp` **pas:**

```
Ingrid Virgo
4587
Engineering
Julia Rich
4588
Finance
```

Programi 16 — Output i `with_multiple.py` II

Shënim

Funksioni nuk ndryshon përmbajtjen e skedarit origjinal, por krijon një të ri me ndryshimet.

Jashtë bllokut `with`

Kujdes

Pasi del nga `with`, objekti i skedarit është **i mbyllur**. Përdorimi i tij më tej shkakton `ValueError: I/O operation on closed file`.

Përfitime kryesore

- Mbyllje automatike dhe e sigurt.
- Më pak kod ndihmës.
- Praktikë e rekomanduar në Python.

Përtej skedarëve

Shënim

`with` përdoret edhe për: lidhje me BD, bllokime *thread*-esh, burime rrjeti, etj.
Çdo objekt me `__enter__()` dhe `__exit__()` mund të jetë context manager.

Përmbledhje e kapitullit

- Nevoja për skedarë dhe hapat *hap—proceso—mbyll*.
- Llojet: tekst/binar; akses: sekuencial/direkt.
- `open()` me `'r'`, `'w'`, `'a'`; rrugë relative/absolute.
- Shkrim/lexim tekst; `\n`, `rstrip()`.
- Numrat: `str()`, `int()`, `float()`.
- Cikle: `while` (EOF) dhe `for` mbi objektin e skedarit.
- Record-e: *name*, *id*, *dept* — rend i qëndrueshëm.
- `with` për mbyllje automatike dhe kod më të sigurt.

Faleminderit!

Pyetje?